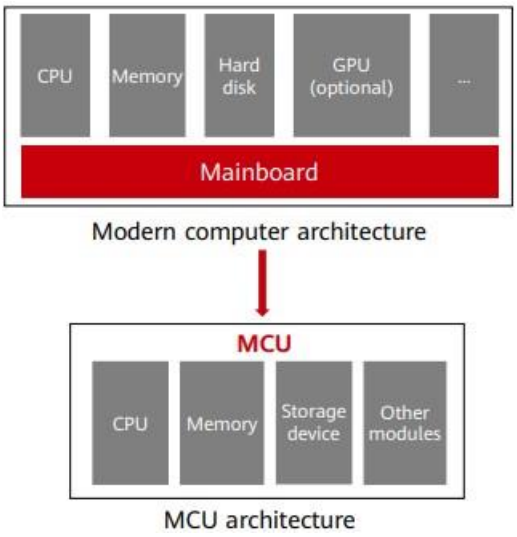
MCU BASIC



MCU architecture is the same as PC architecture. However, the CPU + memory + storage device are now integrated into one chip, the MCU. MCUs greatly simplify the electronics, and enable hardware engineers to design them with less experience. Currently, an MCU performs just as well as earlier computers. Why use an integrated processor? While some components, like I/O, might experience reduced performance upon integration, many other elements exhibit marked improvement due to instantaneous data transmission between processors. Integrated processors have fewer components on the circuit board, simplifying on-chip debugging, reducing board-level faults, consuming less power, shrinking in size, and becoming more cost-effective. However, integrating components into a processor rather than a circuit board limits flexibility in adding, altering, or removing functions within an integrated processor.

MCU development history:

1) Exploration Stage (1976-1978)

   Represented by MCS-48 series MCUs. Their launch started the exploration in the industrial control field. The manufacturing process was backward, the integration technology was immature, and dual chips were used.

2) Upgrade Stage (1978-1982)

   Functional components such as the CPU, parallel interface, timer, RAM, and ROM were integrated into one chip, but the MCU had low performance, few types, and few application scenarios.

3) Full-fledged Stage (1982-1990)

   The 8-bit MCU was fully functional. The storage capability, addressing range, interrupt source, parallel I/O port, and timing/counter were improved. In addition, the full-duplex serial communication interface was integrated into the MCU. The instruction system supported multiplication and division, bit operation, and bit instructions

4) Diversification Stage (1990)

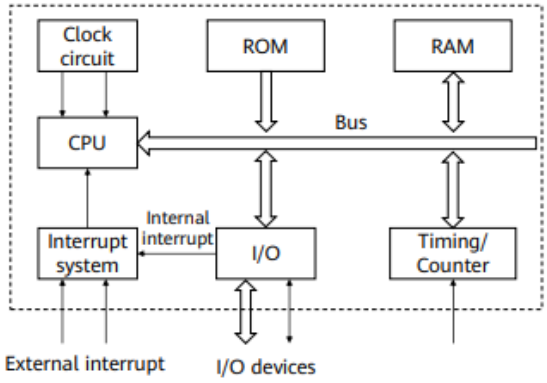
The MCU has been used in various fields. The 8/16/32/64-bit universal MCU with high speed, large addressing range, and high compute has been developed. The 16-bit MCU and 8-bit high performance MCU are more popular.

In the initial stages, an MCU operates as specialized hardware for automated control tasks, like cyclically gathering temperature data and triggering alarms at predefined thresholds, usually accomplished within a single while loop statement. Some MCUs incorporate small OS systems for enhanced utilization. As the MCU integrates with CPUs, RAM, Flash, and connects to external storages like DDR or larger flash storages, particularly in mobile devices, it transforms into an application processing unit (APU). Additional modules such as DSP are embedded to process audio signals, shifting its function from a specialized control unit to a versatile, generalpurpose device. In essence, a non-computer device with computing capabilities is an embedded device.

MCU & IoT

The function of the MCU and the program running on it were very simple: collecting data and reporting an alarm if the data reaches the threshold to achieve automatic control. After the concept of IoT, people began to realize the significance of data networking and data linkage across devices, and started making many landmark applications and products. Multiple MCUs and communication modules form a simple IoT system. Growing IoT also grows MCU
applications

MCU Architecture

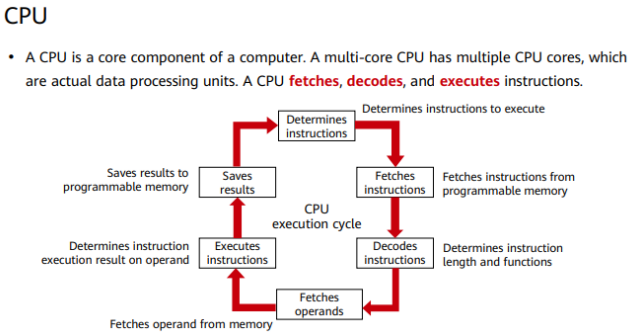


• Generally, the complexity of the main processor determines whether it is a microprocessor or a microcontroller. A microprocessor integrates only a few memory and I/O interfaces, while a microcontroller integrates most of the system memories and I/O interfaces into a chip. However, these traditional definitions do not fit modern processor design. For example, microprocessors can integrate more components.

• An interruption is a signal triggered by an event during the execution of an instruction stream by the main processor. This means that an interruption can be asynchronously triggered by events such as external hardware devices, resets, and power failures, or synchronously triggered by instruction-related

activities such as system calls or invalid instructions. Once such a signal is triggered, the main processor stops executing the current instruction stream and starts to process the interruption.

• Interruption signals are mainly from software, internal hardware, and external hardware.



CPU and system clock

▫ The execution of a processor is ultimately synchronized by an external system or primary clock located on the mainboard. The primary clock consists of an oscillator and some other components. It generates a on/off pulse signal sequence at a fixed frequency. The control unit and some other components on the embedded system board work following the primary clock. The frequency of primary clocks varies, but the frequency must meet the requirements of the slowest component on the circuit board. In some cases, the primary clock can use clock signals at different frequencies generated by other components on the circuit board

Memory

- A CPU needs to fetch data and instructions to be processed from memory.
- Common memory includes read-only memory (ROM) and random access memory (RAM). π The on-chip ROM is the memory integrated into a processor. It stores data and instructions even when the system runs out of power, so ROM is generally used to store programs. π RAM is usually called main memory. All its addresses can be directly accessed (at random, not at a sequence from a specific start point). The data in RAM can be modified multiple times. Unlike ROM, RAM data will be erased if the system runs out of power, so RAM is volatile

Input/Output Interface

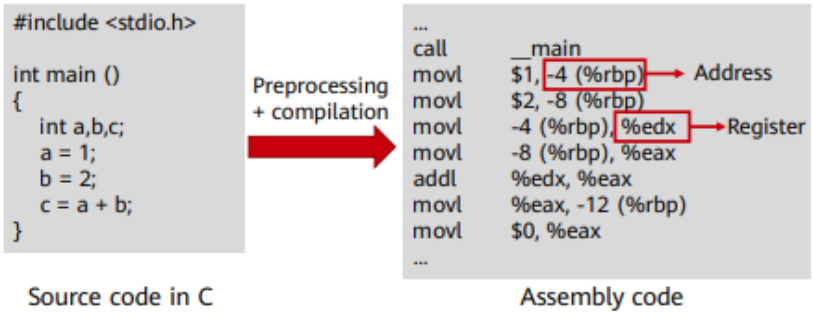
- The input/output interfaces are processor I/O interfaces and onboard I/O interfaces. π Processor I/O interfaces mainly transfer data across other components of the processor, memory on the circuit board, and the external I/O interface of the processor. π The onboard I/O interfaces mainly exchange information between a system board and I/O devices connected to the embedded system.
- For processor I/O interfaces, except integrated circuit (IC) interfaces, basic features are essentially the same as when directly deployed on a circuit board.

Bus

- This mainly refers to the processor bus. A processor bus interconnects the main internal components (such as the CPU, memory, and I/O interface) of the processor, and transmits signals among them.
- A bus can be classified by what it transmits: data bus, address bus, control bus. π Data bus: a bidirectional bus used to transfer data between a processor, memory, and an I/O interface. π Address bus: used to select the memory location the processor reads from or writes to. The data flow of the address bus is unidirectional, from the processor to the memory and I/O interface. π Control bus: consists of multiple independent control/signaling lines. A typ

Instruction Sets

- A computer instruction directs the work of a machine. A program is a series of instructions in a certain sequence. Program execution is the working process of a computer. An instruction set calculates and controls a computer system in a CPU. Each new type of CPU is designed with a series of instruction systems that match other hardware circuits. How advanced an instruction set is depends on CPU performance, making it an important indicator of the CPU performance. is also an important CPU metric.



Common ARM instructions

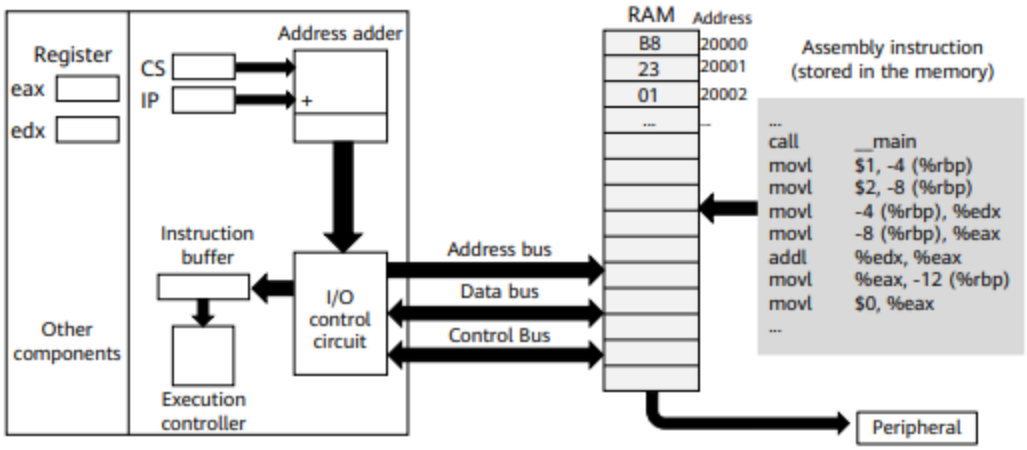
Instruction	Description
MOV	Data transfer
AND	Logic AND
EOR	Exclusive OR
ORR	Logic OR
ADD	Addition
SUB	Subtraction
LDR	Data transfer from memory to register

RISC & CISC

- CPU instruction sets are classified by architecture: complex instruction set computer (CISC) or reduced instruction set computer (RISC) Item CISC R

Item	CISC	RISC
Instruction system	Many instructions	Few instructions (usually less than 100)
Execution duration	Long, especially instructions such as copying the whole content of memory or copying the contents of multiple registers to a memory.	Short
Code length	Variable (max 15 bytes)	Fixed (usually 4 bytes)
Addressing mode	Various	Simple
Operation	Arithmetic and logical operations can be performed on memory and registers.	Arithmetic and logic operations can be performed only on registers.
Compilation	Efficient object code programs generated using optimized compilers are not supported.	Efficient object code programs generated using optimized compilers are not supported.

MCU Working Principle



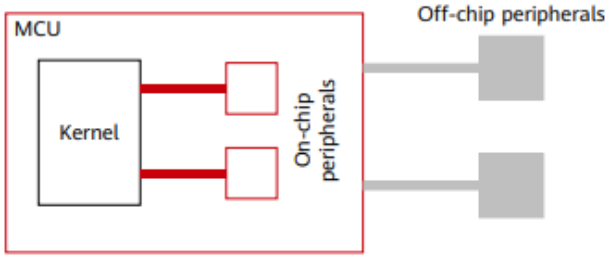
• The initial values of CS and IP are set before the CPU is delivered.

- The memory reads the program from the Flash.
- The CPU obtains the instructions from the specified position in the memory based on the address calculated by the CS and IP registers.
- The instructions are executed in the execution controller, and the intermediate result is stored in an internal register (such as eax and edx).
- After the instructions are executed, the final calculation result is output to the memory.
- The memory transmits the data to the peripheral through the buses to display the result.

5.3. MCU Peripherals

MCU Peripherals

- An MCU peripheral is an MCU external functional module. Examples: analog/digital (A/D) converter, pulse width modulation (PWM), and timer. They are controlled through the I/O, SPI, or I2C bus.
- MCU peripherals can be on-chip and off-chip.



HUAWEI

The material outlines the progression of IC integration techniques, detailing the use of separate peripherals like PWM, ADC, CAN, and DSP chips in the early stages due to limited IC integration. It highlights the current integration of some peripherals, such as PWM and ADC, into DSP chips. The distinction between on-chip and off-chip peripherals is emphasized: on-chip peripherals refer to devices integrated within a chip (like STM32) but independent of its kernel (e.g., serial ports, Flash, I2C, SPI, ADC, DAC), while off-chip peripherals are external devices (e.g., display screens, SD cards) not part of the chip. Understanding this distinction is vital in the development of integrated systems, like microcontrollers, to differentiate between internally integrated devices and those requiring external connections.

GPIO

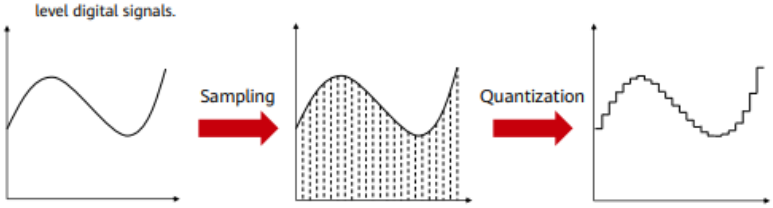
- GPIO Controller Management: GPIO pins are managed by a GPIO controller in groups. Each group of pins is associated with one or more registers. The GPIO controller handles these pins by interacting with these registers, facilitating the reading and writing of data to control the pins.
- Pin Functionality: GPIO pins can perform several functions such as outputting high or low levels of electrical signals and reading the status of the pin (high or low level). This flexibility allows them to adapt to various tasks.
- Versatile Usage: These pins serve multiple purposes in hardware interactions. They can be employed to exchange data with hardware (e.g., UART communication), control hardware components like LEDs and buzzers, and monitor hardware status signals such as interrupt signals.

ADC

- ADC's Role: ADCs are essential in the digital realm as they convert real-world analog signals (like temperature, pressure, or sound) into digital signals, facilitating their processing within digital systems. This conversion makes it easier to manipulate and analyze analog data in digital systems.
- Conversion Process: ADCs typically transform analog quantities through an analog comparator into binary discrete signals. To perform this conversion, an ADC necessitates a reference analog quantity, often the maximum convertible signal size. The resulting output digital quantity represents the input signal's size relative to the reference signal size.
- DAC's Function: Conversely, a digital-to-analog converter (DAC) operates in the opposite direction. While an ADC converts analog signals to digital ones, a DAC translates digital signals within a microcontroller unit (MCU) into analog signals. This capability allows digital systems to interact with and affect the physical world by generating analog signals based on digital inputs.

ADC Working Principles

- Analog-to-digital conversion includes **sampling**, **quantization**, and **encoding**.
  - Sampling: samples values of a continuous input signal at discrete intervals in time.
  - Quantization: replaces a continuous analog signal with a limited number of discrete values at a certain interval.
  - Encoding: represents quantized values using binary values and then converts those values into binary or multi-level digital signals.



HUAWEI

PWM

- **Pulse Width Modulation (PWM)** modulates the width of a series of pulses to obtain the required waveform (including the shape and amplitude) and performs digital coding on the analog signal level by adjusting the duty cycle to changes of signals and energy. The duty cycle **is the percentage of the time during which a signal or system has high voltage.** For example, the duty cycle of a real square wave is 50%.



Square wave with 50% duty cycle

$$Duty\ Cycle = \frac{T_{HL}}{T} \times 100\%$$

Square wave with 25% duty cycle

HUAWEI

Timer

- This important MCU module is widely used in the detection and control fields. Developers often use a timer for time-related operations.
- An MCU timer works as a counter of the number of times a pulse reaches a rising or falling edge of a square wave signal.



HUAWEI

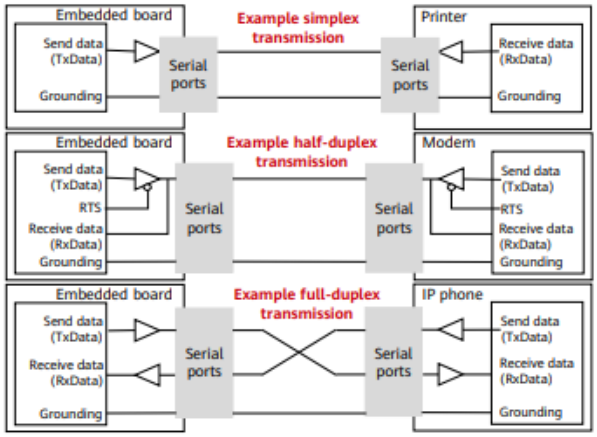
Serial nPort Bus Overview

Serial I/O involves transmitting data bit by bit over a single cable, offering cost efficiency but slower transmission rates. It's categorized based on direction: simplex (one-way), half-duplex (bidirectional, not simultaneous), and full-duplex (simultaneous bidirectional). Transmission can be synchronous (regular intervals synced to a CPU clock) or asynchronous (intermittent, irregular intervals).

Parallel transmission refers to transmit multiple bits of data simultaneously. It is not described in this course.

Simplex, Half-Duplex, and Full-Duplex

- Data can be transmitted between two devices unidirectionally (simplex), time-division bidirectionally (half-duplex), or simultaneously bidirectionally (full-duplex).
  - Simplex: data streams can be transmitted or received only in one direction.
  - Half-duplex: data streams can be transmitted and received bidirectionally, but only in one direction at a time.
  - Full-duplex: data streams can be transmitted and received in any direction at the same time.

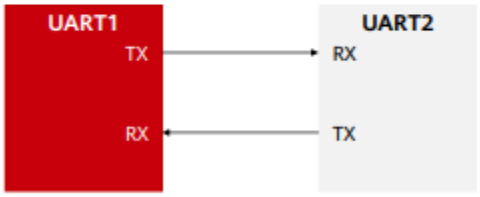


HUAWEI

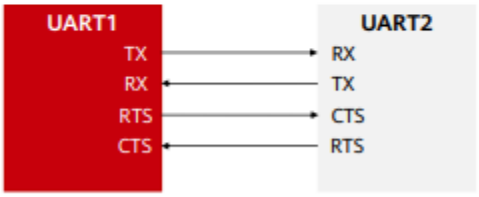
UART

UART | I2C | SPI

- Universal Asynchronous Receiver/Transmitter (UART) is a universal serial data bus used for **asynchronous communication** in **full-duplex** mode.
- UART is widely used to print information for debugging or to connect to various peripherals such as GPS and Bluetooth.
- UART has different interface standards and bus standards: RS-232, RS-499, RS-423, RS-422, and RS-485. Their main difference is the level range.



2-wire UART connection



4-wire UART connection

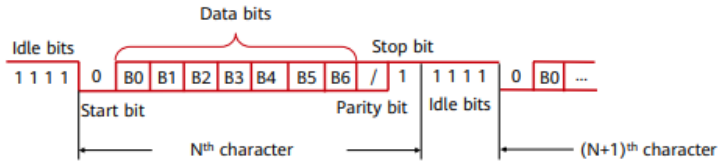
HUAWEI

- TX: UART transmitter. It is connected to the RX of the peer UART.
- RX: UART receiver. It is connected to the TX of the peer UART.
- RTS: Request to Send signal, indicating whether the local UART is ready to receive data. It is connected to the CTS of the peer UART.
- CTS: Clear to Send signal, indicating whether the local UART is allowed to send data to the peer end. It is connected to the RTS of the peer UART.

UART Frame Structure

UART | I2C | SPI

- The UART transmitter and receiver must have the same settings for attributes such as **baud rate** and **data format (start bit, data bits, parity bit, and stop bit)** before they start to communicate. A UART sends data to the peer end over the TX and receives data from the peer end over the RX. When the buffer used by a UART for storing received data reaches a preset threshold size, the Request to Send (RTS) signal of the UART changes to the data cannot be received, and the peer UART is prevented from sending data by the Clear to Send (CTS) signal.



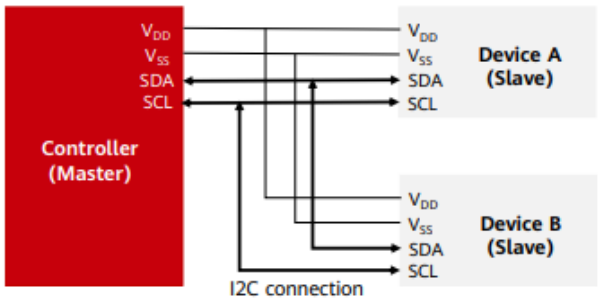
HUAWEI

- Start bit
- Data bits
- Parity bit
- Stop bit
- Idle bits
- Baud rate

I2C

UART | I2C | SPI

- The Inter-Integrated Circuit (I2C) is a simple, bidirectional, two-wire, and synchronous serial bus developed by Philips.
- With I2C, one controller communicates with one or more devices through the serial data line (SDA) and serial clock line (SCL).



HUAWEI

VCC powers the circuit, while VDD is the chip's operating voltage. The I2C module offers APIs for managing controllers, transferring data through unique node addresses, and enabling communication between controllers and devices. This communication is serial, synchronous, at low levels, and low rates, transmitting data in 8-bit serial mode over SDA lines, synchronizing clocks between parties, using level signals due to close proximity, and operating at low kHz rates for inter-IC transmission with limitations set by device capabilities.

I2C Communication Timing (1)

UART | I2C | SPI

- I2C timing consists of four elements: **START condition**, **STOP condition**, **acknowledge (ACK) condition (0)**, and **not-acknowledge (NACK) condition (1)**.
  - START condition: a high-to-low voltage transition on the SDA line while the SCL voltage is high. It is a level transition timing signal, not a level signal.
  - STOP condition: a low-to-high voltage transition on the SDA line while the SCL voltage is high. It is a level transition timing signal, not a level signal.

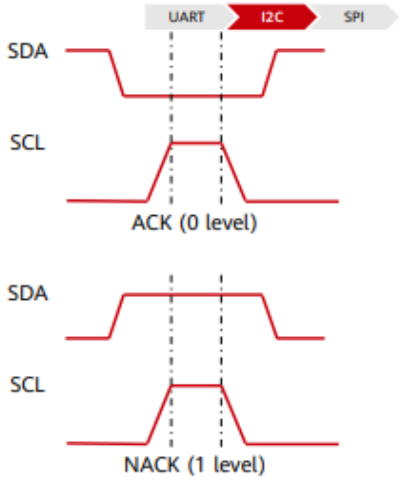


HUAWEI

I2C Communication Timing (2)

UART | I2C | SPI

- Response signals:
  - All data on the I2C bus is transmitted in **8**-bit bytes. Each time the sender transmits a byte, the data line is released during the clock pulse, and the receiver feeds back a response signal.
  - When the response signal is low, it is defined as an ACK (the receiver successfully received the byte). When the response signal is high, it is defined as an NACK (the receiver failed to receive the byte).

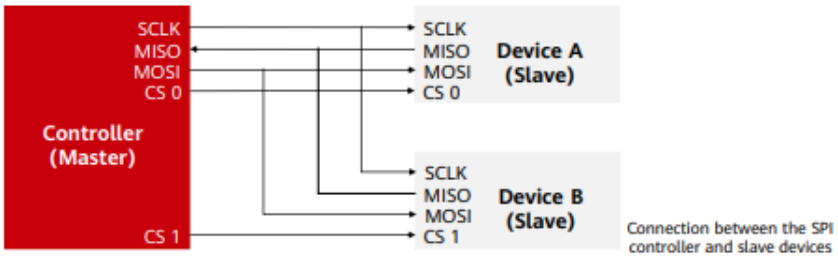


HUAWEI

SPI

UART I2C SPI

- Serial Peripheral Interface (SPI) is a high-speed, full-duplex, and synchronous communication bus developed by Motorola.
- It is commonly used for communication with flash memory, real-time clocks, sensors, and ADCs.
- The figure below shows the connection between one controller and two slave devices (device A and device B). Device A and device B share three pins (Serial Clock (SCLK), Master In Slave Out (MISO), and Master Out Slave In (MOSI)) of the controller. CS 0 of device A and CS 1 of device B are connected to CS 0 and CS 1 of the controller, respectively.



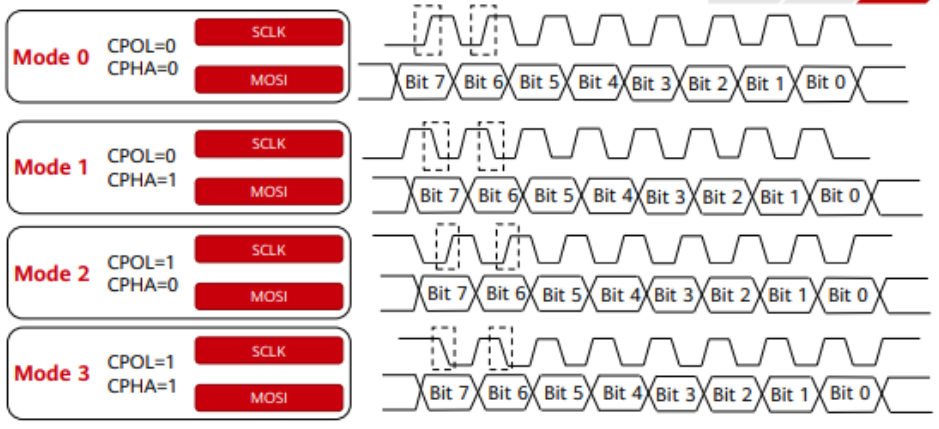
Connection between the SPI controller and slave devices

HUAWEI

1. SPI works in master/slave mode. Generally, there is one SPI controller that controls one or more SPI slave devices. They are connected via four wires:
   ▫ SCLK: clock signal output from the SPI controller
   ▫ MOSI: data output from the SPI controller to a device
   ▫ MISO: data output from an SPI device to the controller
   ▫ Chip select (CS): output from the SPI controller to indicate that data is being sent. It is controlled by the SPI controller.
2. SPI communication is usually initiated by the controller and is performed as follows:
   ▫ The SPI controller selects a device to communicate on the select line.
   ▫ SCLK provides clock signals to the selected device.
   ▫ The SPI controller sends data to the device via MOSI, and receives data from the devices via MISO

SPI Data Transmission

UART I2C SPI



HUAWEI

1. During SPI data transmission, you need to determine the following:
   - ▫ Whether data is sampled on the rising edge or falling edge of the clock.
   - ▫ Whether the idle status of the clock is at a high level or low level.
2. SPI can work in one of the following modes according to the combination of Clock Polarity (CPOL) and Clock Phase (CPHA) of the clock signal:
   - ▫ If both CPOL and CPHA are 0, the clock signal level is low in the idle state
   - and data is sampled on the first clock edge (rising edge).
   - ▫ If CPOL is 0 and CPHA is 1, the clock signal level is low in the idle state and
   - data is sampled on the second clock edge (falling edge).
   - ▫ If CPOL is 1 and CPHA is 0, the clock signal level is high in the idle state
   - and data is sampled on the first clock edge (falling edge).
   - ▫ If both CPOL and CPHA are 1, the clock signal level is high in the idle state
   - and data is sampled on the second clock edge (rising edge).

Comparison Between UART, I2C, and SPI

Item / Serial Port	Number of Signal Cables	Device Subordinate Relationship	Communication Mode	Communication Rate
UART	TX, RX (CTS, RTS)	/	Full-duplex	100 bit/s–1.152 Mbit/s
I2C	SDA, SCL	There are master and slave devices. The I2C selects the slave device by address.	Half-duplex	100 kbit/s (standard) 400 kbit/s (fast) 3.4 Mbit/s (high-speed)
SPI	SCLK, MOSI, MISO, CS	There are master and slave devices. The SPI selects the slave device by CS signal.	Full-duplex communication	Up to dozens of Mbit/s